twist the adage, “Those who cannot remember the past are unable to repeat it.”

B.2 Architectural principles

Event sourcing, command query responsibility segregation (CQRS), and domain-driven design are ideas you can add to your toolbelt that will bring you great benefits as a developer or architect. Let’s take a quick look at them.

B.2.1 Event-driven systems

If you capture state changes in an incremental, append-only way, you can build event-driven architectures. In these systems, components communicate via events. From this perspective, an event can be defined as data that refers to a fact, and it is used to communicate with another process.

Events are related to messages, but they have subtle differences. An event is a message sent to a destination whose consumers are unknown. A message, on the other hand, identifies the target with its consumer. All events are messages, but not all messages are events; a message, for example, can also be used as a command or a response.

The fact that events are messages gives event-driven systems the properties of reactive systems, and as such, they are isolated in time, space, and failure. But there are differences here as well. In message-driven systems, a component can send messages to multiple addresses, whereas in an event-driven system, messages